

Bootcamp: Full Stack Junior

Modulo 3: Backend con PHP y Laravel

Semana 9

Contenido

1

Hashing de contraseñas y manejo de secretos en PHP.

2

Protocolo JWT (JSON Web Tokens) en APIs.

3

Gestión de Roles y Permisos (Roles y Políticas).

Contenido:

1. Hashing de Contraseñas y Manejo de Secretos en PHP

1.1. ¿Por qué no se almacenan contraseñas en texto plano?

Almacenar una contraseña tal cual la escribe el usuario ("123456") es el equivalente digital a dejar la llave de una bóveda pegada a la puerta con cinta adhesiva.

Riesgos de seguridad

- **Brechas de Seguridad (Data Breaches):** Si un atacante logra acceso a la base de datos (vía SQL Injection o acceso al servidor), tener las contraseñas en texto plano le da acceso inmediato a todas las cuentas.
- **Dumps de Base de Datos:** Los archivos de respaldo (backups) suelen ser objetivos de ataques. Un archivo .sql expuesto con contraseñas legibles compromete no solo tu app, sino la vida digital del usuario.
- **Reutilización de Contraseñas:** El mayor peligro es que la mayoría de los usuarios repiten la misma contraseña en múltiples servicios (banco, correo, redes sociales). Una filtración en tu plataforma puede causar un efecto dominó catastrófico.

Responsabilidad Legal y Buenas Prácticas

- **Cumplimiento (Compliance):** Regulaciones como el **GDPR** (Europa) o la **Ley de Protección de Datos Personales** exigen medidas técnicas adecuadas para proteger la información sensible. El texto plano es una negligencia legal.
- **Estándares de la Industria:** Organizaciones como **OWASP** dictan que el hashing es el estándar mínimo. No seguirlo puede invalidar seguros contra ciberataques o certificaciones de seguridad.

Principio de Mínimo Conocimiento

Este principio establece que un sistema solo debe conocer la información estrictamente necesaria para operar.

- **Tú no necesitas saber la contraseña:** Solo necesitas saber si la cadena que el usuario ingresa **coincide** con la que registró. El hashing permite verificar la identidad sin que el servidor jamás "vea" o "conozca" el secreto original.



Ilustración 1 - Flujo de seguridad de contraseñas - Fuente: GPT Image

1.2. Hashing en PHP y Laravel 12

En PHP moderno, ya no se usan funciones antiguas como `md5()` o `sha1()` para contraseñas, ya que son vulnerables a ataques de fuerza bruta mediante GPUs modernas.

El estándar: Bcrypt y Argon2

Laravel 12 utiliza por defecto **Bcrypt**, pero soporta **Argon2id** (el ganador de la Password Hashing Competition).

Concepto clave: El *Hashing* es una función de una sola vía. A diferencia del cifrado, no se puede "descifrar".

Diagrama de Hash de Contraseña

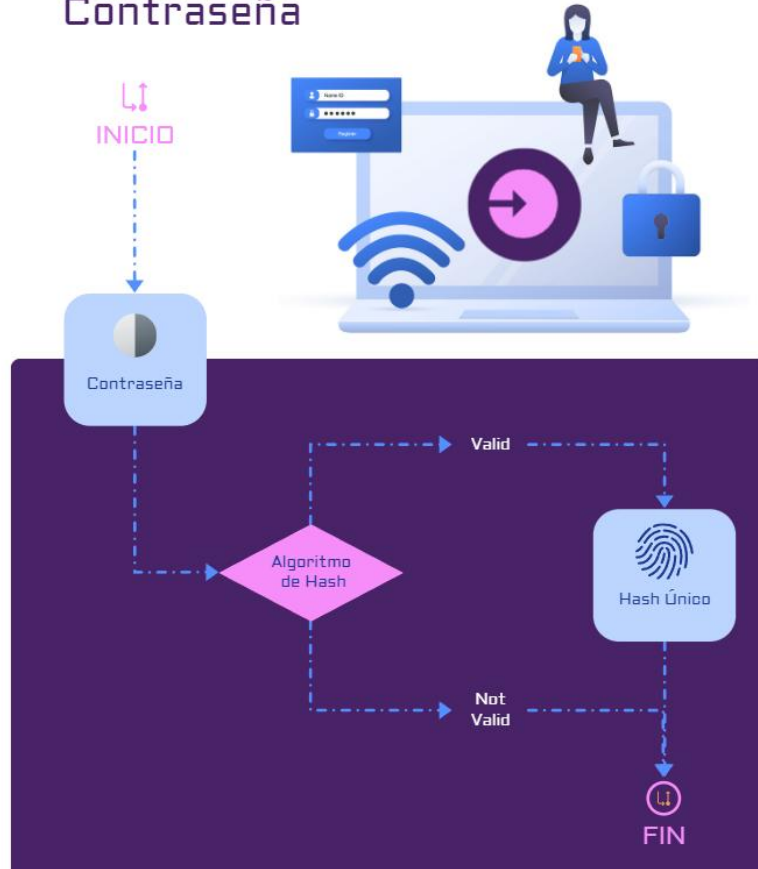


Ilustración 2 - Seguridad de contraseñas Laravel 12+ - Fuente: Autor de contenidos

Hashing seguro en PHP y Laravel

Tabla Comparativa de Algoritmos			
ALGORITMO	SEGURIDAD	PERFORMANCE	RECOMENDACIÓN
bcrypt	Alta	Media	✓ Uso general
Argon2	Muy alta	Configurable	✓ APIs modernas
MD5/SHA1	✗ Baja	Alta	✗ No usar

Ilustración 2 - Algoritmos de hash PHP y Laravel 12+ - Fuente: Autor de contenidos

2. Protocolo JWT (JSON Web Tokens) en APIs

Un JSON Web Token es un contenedor compacto y autónomo para transmitir información de forma segura entre partes como un objeto JSON.

Autenticación Stateless (Sin estado)

A diferencia de las aplicaciones web tradicionales, una API "stateless" no guarda información del usuario en la memoria del servidor (sesiones).

- **El Token es la verdad:** Toda la información necesaria para identificar al usuario está **dentro** del token.
- **Independencia:** El servidor no necesita consultar una tabla de sesiones o Redis en cada petición para saber quién es el usuario.

Diferencia: Sesiones Tradicionales vs. JWT

Característica	Sesiones (Stateful)	JWT (Stateless)
Almacenamiento	Lado del servidor (File, DB, Redis).	Lado del cliente (LocalStorage, Cookies).
Escalabilidad	Difícil (requiere sesiones compartidas).	Excelente (cualquier servidor valida el token).
Acoplamiento	El cliente y el servidor deben estar vinculados.	El cliente puede hablar con múltiples microservicios.

Tabla 1 - Sesiones vs JWT - Fuente: Autor de Contenidos

Casos de uso reales

1. **SPAs (Single Page Applications):** Aplicaciones en Vue, React o Angular que consumen una API de Laravel de forma asíncrona.
2. **Mobile Apps:** Las apps nativas (iOS/Android) no manejan cookies de la misma forma que un navegador; los tokens son su lenguaje natural.
3. **Microservicios:** Un servicio de "Pagos" puede validar un token generado por el servicio de "Usuarios" sin tener que consultar la base de datos de este último, gracias a que comparten la misma llave de firma.

2.1. Anatomía de un JSON Web Token

Un JWT se divide en tres partes separadas por puntos (.): header.payload.signature.



Ilustración 3- Anatomía de un JWT - Fuente: Fusionauth

1. **Header (Encabezado):** Indica cómo se debe procesar el token. Típicamente contiene el tipo de token (JWT) y el algoritmo de firma (como HS256 o RS256).

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

Ilustración 4 - Header JWT - Fuente: Autor de contenidos

2. Payload (Cuerpo)

Contiene los **Claims** (afirmaciones). Son piezas de información sobre el usuario y metadatos:

- **sub (Subject):** ID del usuario.
- **iat (Issued At):** Cuándo se creó.
- **exp (Expiration):** Cuándo vence (crucial en Laravel para la seguridad).
- **Custom claims:** Roles, permisos o nombre de usuario.

3. Signature (Firma)

Es la parte más crítica. Se crea tomando el header y el payload codificados, y firmándolos con una **llave secreta** que solo el servidor de Laravel conoce.

- Si alguien cambia un solo bit en el payload, la firma dejará de coincidir y el servidor rechazará el token.

Concepto clave: El Payload NO es privado

Es un error común pensar que los datos en el payload están ocultos. El payload está

simplemente codificado en **Base64Url**. Cualquier persona que intercepte el token puede ver tu ID, tu correo o tus roles usando herramientas como jwt.io. **Regla de oro:** Jamás pongas contraseñas o información sensible (como números de tarjeta) dentro del payload de un JWT.



Ilustración 5 - Mapa mental JWT en Laravel 12+ - Fuente: Autor de contenidos

En Laravel 12, si decides usar JWT puro (en lugar de Sanctum), la librería líder sigue siendo `tymon/jwt-auth`, la cual se integra con el sistema de *Guards* de autenticación del framework.

3. Gestión de Roles y Permisos (Gates y Policies).

3.1. Autenticación vs Autorización

Concepto	¿Qué valida?	Ejemplo
Autenticación	Quién eres	Login
Autorización	Qué puedes hacer	Acceso a recurso

En Laravel 12, la autorización se ha refinado para ser más intuitiva, separando la lógica de negocio de las reglas de acceso. El sistema RBAC (Role-Based Access Control) se implementa principalmente a través de dos mecanismos: Gates y Policies.

3.2. Gates vs. Policies: ¿Cuándo usar cada uno?

Ambos sirven para autorizar acciones, pero su organización es distinta:

- **Gates (Puertas):** Son cierres (closures) basados en funciones. Se usan para acciones que **no** están necesariamente ligadas a un modelo específico (ej. "entrar al panel de administración").
- **Policies (Políticas):** Son clases que organizan la lógica de autorización alrededor de un **modelo de Eloquent** específico (ej. "quién puede editar un Post").

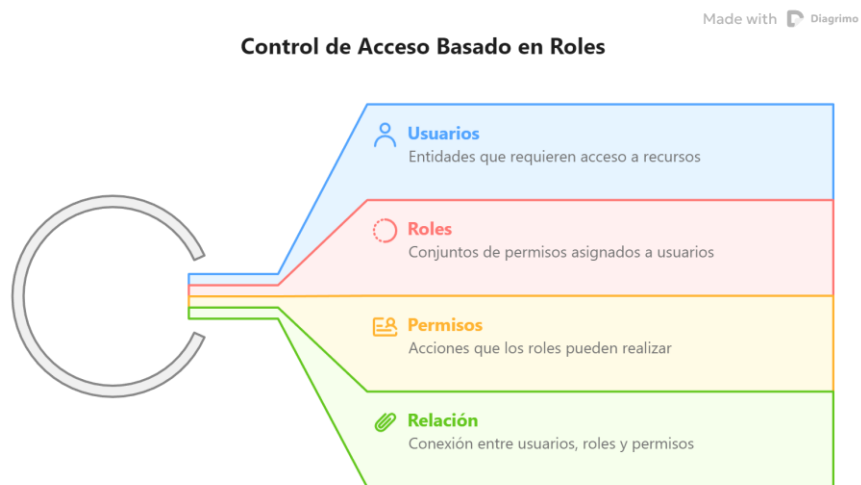


Ilustración 6 - Diagrama conceptual RBAC - Fuente: Autor de contenidos

Recursos de Apoyo



Guia

[Guía de buenas prácticas en Laravel 12+](#)



Material de Lectura

[Blog de últimas novedades de Laravel](#)



Material de Lectura

[Librería de decodificación de tokens](#)